

PSI-Innerproduct의 RsCpsi 및 B2A-OT 구현 정리

본 문서는 새로 추가한 PSI_Innerproduct 구현을 설명한다. 현재 구현은 RsCpsi를 이용해 sender의 associated data를 교집합 위치 기준으로 정렬하고, B2A-OT를 통해 Boolean share가 곱해진 $\mathbb{Z}_p$ arithmetic share를 생성한다. 최종 output은 각 bin i 에 대해 다음을 만족한다.

$$a_i^S + a_i^R = b_i \cdot D_i^Y \pmod{p}.$$

여기서 b_i 는 해당 위치가 교집합에 속하는지를 나타내는 indicator이고, D_i^Y 는 sender가 가진 associated data이다. 즉 교집합 위치에서는 associated data share가 유지되고, 교집합이 아닌 위치에서는 결과가 0이 되도록 만든다.

추가 파일과 역할

파일	역할
volePSI/PSI_Innerproduct.h	wrapper class와 B2A helper API 선언
volePSI/PSI_Innerproduct.cpp	IKNP OT 기반 B2A-OT 및 $\mathbb{Z}_p$ 연산 구현
volePSI/CMakeLists.txt	PSI_Innerproduct.cpp 빌드 대상 추가

전체 구조

Sender는 식별자 집합 Y 와 associated data D^Y 를 입력하고, receiver는 식별자 집합 X 를 입력한다. 두 참여자는 먼저 prime mode의 RsCpsi를 수행한다. RsCpsi는 sender의 associated data를 receiver의 cuckoo bin 기준으로 정렬된 share로 만든다. 이 단계 이후 각 bin i 에 대해 양쪽은 다음 값을 가진다.

$$b_i = b_i^S \oplus b_i^R.$$

$b_i = 1$ 이면 해당 bin은 교집합 원소에 대응하고, $b_i = 0$ 이면 교집합이 아닌 위치이다. 또한 prime mode RsCpsi의 value output은 sender associated data의 additive share로 해석된다. 교집합 위치에서는 다음 관계가 성립한다.

$$D_i^Y = w_i^S + w_i^R \pmod{p}.$$

따라서 필요한 값은 단순히 D_i^Y 의 share가 아니라 indicator가 곱해진 값이다.

$$b_i \cdot D_i^Y = b_i \cdot (w_i^S + w_i^R) \pmod{p}.$$

이를 위해 구현은 B2A-OT를 두 번 수행한다. 첫 번째 B2A-OT는 sender가 가진 w_i^S 에 대해 $b_i w_i^S$ 의 arithmetic share를 만든다. 두 번째 B2A-OT는 receiver가 가진 w_i^R 에 대해 $b_i w_i^R$ 의 arithmetic share를 만든다. 마지막으로 두 결과를 $\mathbb{Z}_p$ 에서 더해 $b_i D_i^Y$ 의 share를 만든다.

Config 구조

PsiInnerproductConfig는 field와 길이 정보를 관리한다. 기본 prime은 RsCpsi prime mode와 동일한 4294967291이다.

```

1 struct PsiInnerproductConfig
2 {
3     u64 mPrime = RsCpsiDefaultPrime;
4     u64 mStatSecParam = 40;
5     u64 mNumThreads = 1;
6
7     u64 dataByteLength() const { return RsCpsiDataByteLength(mPrime); }
8     u64 shareByteLength() const { return RsCpsiPrimeByteLength(mPrime); }
9 };

```

기본 설정에서는 input associated data가 16 bit 이고, arithmetic share는 32 bit 이다. 따라서 sender input associated data는 2 byte로 저장되고, RsCpsi 및 B2A 이후의 share는 4 byte로 저장된다. 내부 계산에서는 u64를 사용하지만 matrix에는 필요한 byte length만 기록한다.

항목	기본값	의미
mPrime	4294967291	연산 field $\mathbb{Z}_p$
dataByteLength()	2 byte	sender associated data 입력 길이
shareByteLength()	4 byte	CPSI/B2A arithmetic share 길이
mStatSecParam	40	RsCpsi security parameter
mNumThreads	1	protocol thread 수

RsCpsi Wrapper 구현

PsiInnerproductSender와 PsiInnerproductReceiver는 내부적으로 각각 RsCpsiSender, RsCpsiReceiver를 가진다. 초기화할 때 ValueShareType::prime을 사용하도록 고정하였다. 따라서 wrapper는 XOR mode나 add32 mode가 아니라, inner product 계산에 필요한 $\mathbb{Z}_p$ share를 얻는 경로만 사용한다.

Sender 쪽 input은 span<block> identifiers와 span<const u64> associatedData이다. associated data는 dataByteLength() 만큼 matrix에 기록한 뒤 RsCpsi에 전달한다.

```

1 oc::Matrix<u8> values(associatedData.size(), mConfig.dataByteLength());
2 setPsiIpPrimeValues(values, associatedData, mConfig.dataByteLength());
3 co_await mCpsi.send(identifiers, values, share, chl);

```

Receiver 쪽은 associated data 없이 identifier만 입력한다. RsCpsi receiver output에는 receiver의 bin mapping, Boolean flag share, 그리고 sender value share의 receiver 몫이 들어 있다.

```

1 co_await mCpsi.receive(identifiers, share, chl);

```

B2A-OT 목표

B2A-OT의 입력은 한쪽이 가진 value $x \in \mathbb{Z}_p$ 와 양쪽이 가진 Boolean share b^{owner}, b^{choice} 이다. 두 share의 XOR가 실제 bit이다.

$$b = b^{owner} \oplus b^{choice}.$$

목표는 두 참여자가 $z^{owner}, z^{choice} \in \mathbb{Z}_p$ 를 얻고 다음을 만족하게 하는 것이다.

$$z^{owner} + z^{choice} = b \cdot x \pmod{p}.$$

현재 구현에서는 libOTe의 IKNP 1-out-of-2 OT extension을 사용한다. value x 를 가진 쪽이 OT sender가 되고, 자기 Boolean share를 선택 bit로 사용하는 쪽이 OT receiver가 된다.

함수	역할
psiIpB2aValueOwner	value x 를 가진 쪽, OT sender 역할
psiIpB2aChoiceOwner	선택 bit를 가진 쪽, OT receiver 역할
psiIpAddPrimeShares	두 B2A 결과를 $\mathbb{Z}_p$ 에서 더함

B2A-OT 메시지 구성

Value owner는 각 row와 각 field element마다 랜덤 $r \leftarrow \mathbb{Z}_p$ 를 샘플링한다. 그리고 자기 arithmetic share를 r 로 둔다. 그러면 choice owner가 받아야 하는 값은 실제 bit b 에 따라 다음과 같다.

$$b = 0 : -r, \quad b = 1 : x - r.$$

하지만 value owner는 choice owner의 Boolean share를 모르므로 실제 b 를 직접 알 수 없다. 그래서 value owner는 자기 Boolean share b^{owner} 에 따라 OT message 순서를 바꾼다.

$$b^{owner} = 0 : (m_0, m_1) = (-r, x - r)$$

$$b^{owner} = 1 : (m_0, m_1) = (x - r, -r)$$

Choice owner는 자기 Boolean share b^{choice} 를 OT choice bit로 사용한다. 그러면 네 가지 경우는 다음과 같이 정리된다.

b^{owner}	b^{choice}	실제 b	choice owner가 받는 값
0	0	0	$-r$
0	1	1	$x - r$
1	0	1	$x - r$
1	1	0	$-r$

즉 선택된 메시지는 항상 다음과 같다.

$$(b^{owner} \oplus b^{choice}) \cdot x - r.$$

따라서 value owner의 share r 와 choice owner의 share를 더하면 다음 식이 된다.

$$r + ((b^{owner} \oplus b^{choice}) \cdot x - r) = (b^{owner} \oplus b^{choice}) \cdot x \pmod{p}.$$

두 번의 B2A-OT가 필요한 이유

RsCpsi 이후 sender와 receiver는 각각 CPSI value share를 하나씩 가진다. 이를 w_i^S, w_i^R 라고 하면 교집합 위치에서

$$D_i^Y = w_i^S + w_i^R \pmod{p}$$

이다. 따라서 전체 D_i^Y 에 indicator b_i 를 곱하려면 다음 두 값을 모두 share 형태로 만들어야 한다.

$$b_i w_i^S, \quad b_i w_i^R.$$

첫 번째 B2A에서는 sender가 value owner가 되고 receiver가 choice owner가 된다. 결과를 (α_i^S, α_i^R) 라고 하면 다음을 만족한다.

$$\alpha_i^S + \alpha_i^R = b_i \cdot w_i^S \pmod{p}.$$

두 번째 B2A에서는 receiver가 value owner가 되고 sender가 choice owner가 된다. 결과를 (β_i^S, β_i^R) 라고 하면 다음을 만족한다.

$$\beta_i^S + \beta_i^R = b_i \cdot w_i^R \pmod{p}.$$

각 참여자는 자신이 가진 두 share를 더한다.

$$a_i^S = \alpha_i^S + \beta_i^S \pmod{p}, \quad a_i^R = \alpha_i^R + \beta_i^R \pmod{p}.$$

따라서 최종적으로 다음이 성립한다.

$$\begin{aligned} a_i^S + a_i^R &= b_i \cdot w_i^S + b_i \cdot w_i^R \\ &= b_i \cdot (w_i^S + w_i^R) \\ &= b_i \cdot D_i^Y \pmod{p}. \end{aligned}$$

Wrapper에서의 실행 순서

Sender 쪽 wrapper는 먼저 RsCpsi sender를 실행한다. 그 후 sender가 가진 value share에 대해 value owner 역할로 B2A를 수행하고, receiver가 가진 value share에 대해서는 choice owner 역할로 B2A를 수행한다.

```

1 co_await send(identifiers, associatedData, share, chl);
2 co_await psiIpB2aValueOwner(share.mFlagBits, share.mValues,
3   ownedValueShare, mConfig, mPrng, chl);
4 co_await psiIpB2aChoiceOwner(share.mFlagBits, share.mValues.rows(),
5   share.mValues.cols(), receivedValueShare, mConfig, mPrng, chl);
6 psiIpAddPrimeShares(ownedValueShare, receivedValueShare,
7   arithmeticShare, mConfig);

```

Receiver 쪽 wrapper는 같은 두 B2A를 반대 역할 순서로 수행한다.

```

1 co_await receive(identifiers, share, chl);
2 co_await psiIpB2aChoiceOwner(share.mFlagBits, share.mValues.rows(),
3   share.mValues.cols(), receivedValueShare, mConfig, mPrng, chl);
4 co_await psiIpB2aValueOwner(share.mFlagBits, share.mValues,
5   ownedValueShare, mConfig, mPrng, chl);
6 psiIpAddPrimeShares(receivedValueShare, ownedValueShare,
7   arithmeticShare, mConfig);

```

구현 세부사항

psiIpB2aValueOwner는 `oc::MatrixView<u8>` 형태의 value share를 입력으로 받는다. matrix의 row는 CPSI output bin에 대응하고, column은 field element를 byte 단위로 저장한다. 기본 설정에서는 row마다 field element 하나가 있고, 각 element는 4 byte이다. 함수는 각 field element에 대해 OT 하나를 생성한다.

psiIpB2aChoiceOwner는 같은 row 수와 value byte length를 입력으로 받는다. mFlagBits는 OT choice vector로 확장된다. row마다 field element가 여러 개라면 같은 Boolean share가 각 element에 반복되어 들어간다.

OT message는 block 타입으로 전달한다. 실제 prime element는 u64로 계산한 뒤 block의 하위 64 bit에 넣는다. 수신 후에는 다시 하위 64 bit를 읽고 p 로 reduction하여 $\mathbb{Z}_p$ 원소로 해석한다.

psiIpAddPrimeShares는 두 B2A 결과 matrix를 element 단위로 읽어서 $\mathbb{Z}_p$ 덧셈을 수행한다. 이 함수의 output이 wrapper에서 반환하는 arithmeticShare이다.

검증

검증용 테스트 `Cpsi_PsiInnerproduct_b2a_test`를 추가하였다. 테스트에서는 일부 원소만 교집합이 되도록 입력을 만들고, B2A 까지 끝난 후 양쪽 arithmetic share를 더해서 다음 값과 비교한다.

$$a_i^S + a_i^R = \begin{cases} D_i^Y & x_i \in Y, \\ 0 & x_i \notin Y. \end{cases}$$

실행 명령은 다음과 같다.

```
1 ./out/build/osx/frontend/frontend -u Cpsi_PsiInnerproduct_b2a_test -n 64 -nt 1
```

테스트 결과는 통과하였다.